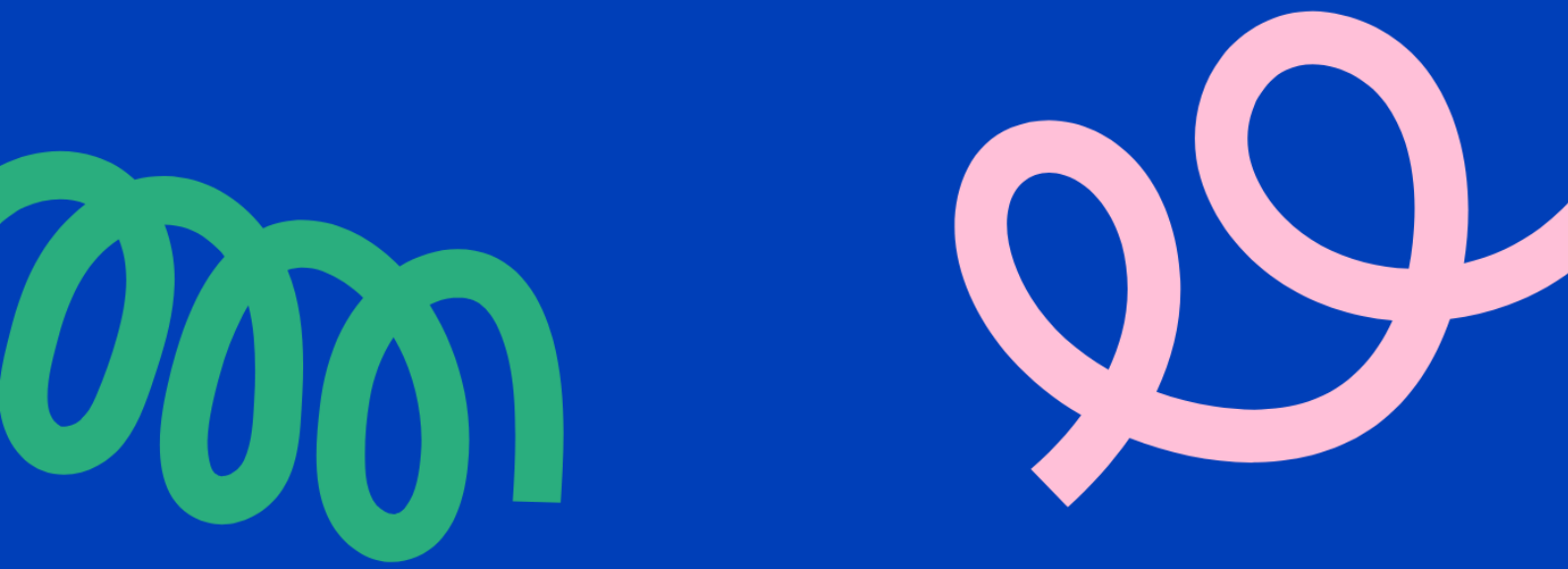
Faculté des Sciences de Bizerte

MlOps Project Report

Reporting Period
JANUARY

Prepared by
Safwen Gharbi | CI3



2026

Table of contents

Introduction

Before starting the report, I want to define the most important terms in the project description :

- Confidence score: A numerical value (a probability) indicating a model's certainty that a retrieved result is correct.
- Semantic index: A searchable data structure that stores embeddings (where data is converted into vector representations). This allows users to find results based on meaning, rather than on exact keyword matches.
- Embedding: A list of numbers (a vector) that represents the meaning of text. For example, if word1 and word2 have similar meanings, their vectors will be mathematically close.
- Similarity score: A metric (in our case, cosine similarity) that measures the distance between two embeddings. A higher score indicates a more similar meaning.
- Lexical search (BM25): A traditional search algorithm that ranks results by word occurrences in a document. (the project implements a hybrid retrieval : semantic + lexical fallback
- Sentence-transformers: A popular Python library used to convert sentences or paragraphs into dense vector embeddings
- FAISS (Facebook AI Similarity Search): A high-performance library developed by Meta for efficiently searching and clustering massive amounts of dense vectors
- ANN index (Approximate Nearest Neighbor): A type of index that speeds up search by finding close matches rather than calculating the exact distance to every single document in the database
- Sigmoid mapping: A mathematical function that squashes any raw score into a probability range between 0 and 1
- Calibration: The process of adjusting a model's output scores so they accurately reflect the true probability of correctness (For example, ensuring a 0.8 score actually means « 80% likely to be correct »)
- Recall@K: A performance metric that answers: "Out of all the relevant answers that exist, how many did we find in our top K results?"
- MRR (Mean Reciprocal Rank): A metric that evaluates the ranking quality : it gives a higher score if the first correct answer appears at the very top of the list rather than lower down

Data Collection

Due to data scarcity for this project (I didn't find an equipment catalog or a queries dataset), I decided to generate both datasets specifically for this semantic matching task using real data

Catalogue dataset

The catalogue dataset is our equipments catalogue , it is a product catalogue of laptops, smartphones, PC components, and car parts, designed to approximate the most popular items in Tunisia between 2023 and 2025.

Queries dataset

The queries dataset is composed of search queries that are semantically mapped to the products in the catalogue, reflecting realistic search behaviour for technology and automotive items in Tunisia over the 2023–2025 period

Note: In this report, I explained all the code and everything I did in the project, along with the experiments and their analysis at the end

API et évaluation du moteur de recherche sémantique

App.py

Let's begin with the main entry of our application , This code defines a small FastAPI web service that exposes the semantic search model over HTTP. It first configures logging and paths, loads application settings and environment variables to locate model artifacts and index versions, then builds a SemanticMatcher instance from those artifacts. It serves a static frontend (index.html) at the root URL, sets up Prometheus instrumentation for monitoring, and defines Pydantic models (MatchRequest, MatchItem, MatchResponse) to validate and structure input/output data. The /health endpoint returns a simple status check, while the /match POST endpoint takes a designation string and top_k, calls matcher.match(), extracts an optional matching mode from the explanation text, logs latency and request details, and finally returns the ranked match results. In case of errors, it logs them with stack traces and re-raises the exception.

evaluation.ipynb

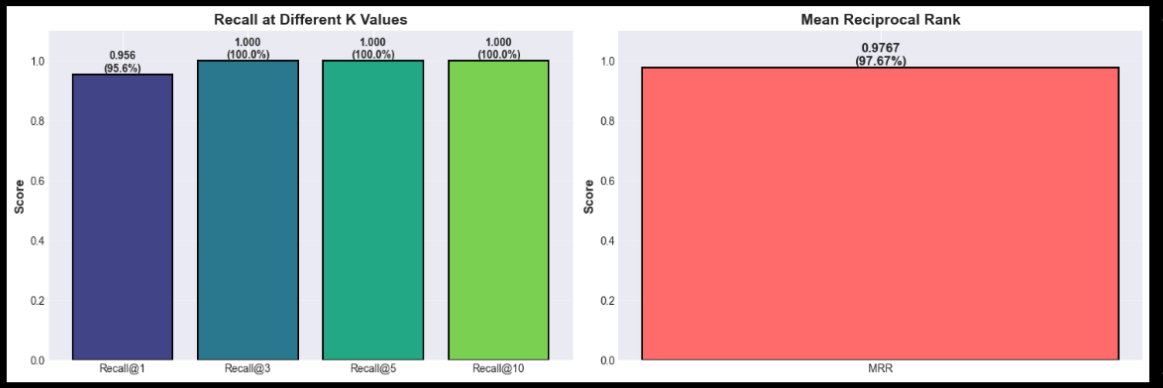
This notebook evaluates how well the semantic matcher retrieves the correct catalog item for a set of test queries, computes global metrics, and visualizes them. It loads query–item pairs from queries.csv, builds a SemanticMatcher from saved artifacts, and for each query runs matcher.match() to collect ranked item IDs, scores, confidences, match mode (hybrid vs lexical), query length, and the rank of the true item if present. Using these ranked lists and ground-truth IDs, it calculates Recall@K for several K values and the Mean Reciprocal Rank (MRR), then plots bar charts for

Recall@K and MRR to show overall retrieval quality. It further groups queries into length bins (For example 1–2 words, 3–4 words) to analyze how performance, scores, and number of queries vary with query length, printing a small table summarizing this. Finally, it prints a detailed text report with dataset statistics, overall recalls and MRR, score and confidence summary statistics, distribution of matching modes, and the percentage of queries where the correct answer appears in the top-1 and top-5 results, giving a compact performance summary of the model.

```
Total queries: 1110
Model: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
Index version: 1.0.0
```

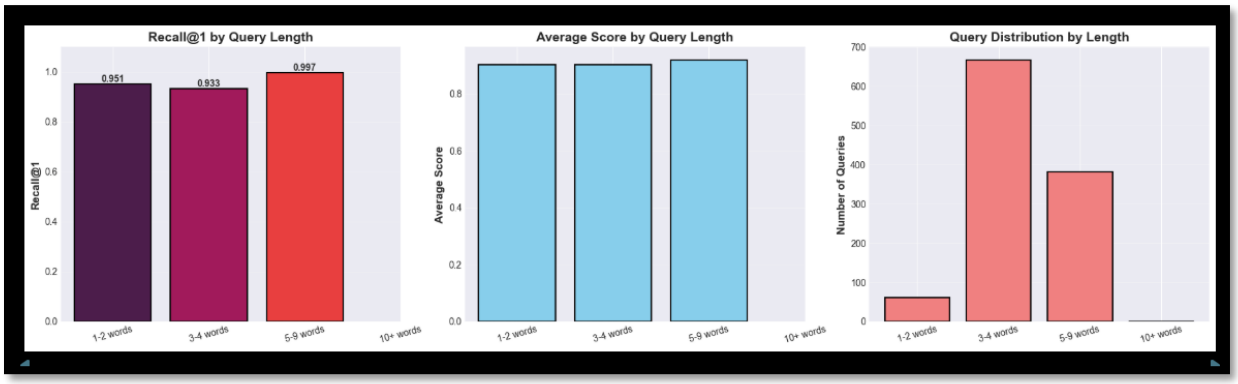
- Model: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2. This is a popular, lightweight model designed to understand sentence meaning across multiple languages. It is optimized for detecting paraphrases, which explains why it is so good at matching user queries to catalog items even if the wording is different
- Index Version: 1.0.0 (The first version of my search index)

```
================================================
PERFORMANCE METRICS
================================================
Recall@ 1: 0.9559 (95.59%)
Recall@ 3: 1.0000 (100.00%)
Recall@ 5: 1.0000 (100.00%)
Recall@10: 1.0000 (100.00%)
MRR:       0.9767
================================================
```



- Recall@1 (95.59%): The correct answer is the first result roughly 95% of the time.
- Recall@3, @5, @10 (100.00%): This is a perfect score. It means that the correct answer always appears within the top 3 results. If a user looks at the first three suggestions, they will find what they are looking for every single time in this dataset.

- MRR (Mean Reciprocal Rank) = 0.9767: This score ranges from 0 to 1. A score of 0.97 is extremely high. It indicates that when the correct answer wasn't the first, it was usually the second.



- Recall@1 by Query Length : This measures how often the correct answer appears as the very first result.
    - 5-9 words: The model performs best here with a near-perfect 99.7% success rate.
    - 1-2 words: Short queries also perform very well at 95.1%.
    - 3-4 words: Even the "lowest" performing category is excellent at 93.3%.
- Average Score by Query Length (Middle Chart): The blue bars show the similarity score (confidence) assigned by the model. The scores are consistently high (around 0.9) across all query lengths, meaning the model is equally confident regardless of whether the query is short or long.
- Query Distribution (Right Chart): This shows what the dataset looks like. The vast majority of queries are 3-4 words long (the red bar), followed by 5-9 words. Very short (1-2) or very long (10+) queries are rare.

```
FINAL EVALUATION SUMMARY
======================================================================

Dataset Statistics:
  Total Queries: 1110
  Total Items in Catalog: 111

Overall Performance:
  Recall@ 1: 0.9559 (95.59%)
  Recall@ 3: 1.0000 (100.00%)
  Recall@ 5: 1.0000 (100.00%)
  Recall@10: 1.0000 (100.00%)
  MRR:       0.9767

Score Statistics:
  Mean Score:      0.9083
  Median Score:    0.9137
  Std Dev:         0.0372

Confidence Statistics:
  Mean Confidence: 0.8580
  Median Confidence: 0.8644
  Std Dev:         0.0302

Matching Modes:
  Hybrid         : 1110 (100.0%)

Queries with correct answer in top-1: 1061 (95.6%)
Queries with correct answer in top-5: 1110 (100.0%)
```

This provides the raw statistics and details behind the charts.

- Dataset Stats: The test was run on 1,110 queries searching against a catalog of 111 items.
- Score Statistics:
    - Mean Score (0.9083): The model finds very strong matches (high similarity).
    - Low Std Dev (0.0372): The performance is consistent so there are no wild fluctuations in how well the model matches items.
- Matching Modes: we used a Hybrid search approach (lexical + semantic) for every single query.
- Bottom Line: The summary confirms that 1,061 queries had the correct answer at rank 1, and all 1,110 queries had the correct answer by rank 5.

Cœur du système de recherche sémantique

build_index.py

This script is a command-line utility that builds the semantic and lexical search index artifacts for the semantic matcher system. It first adjusts sys.path so it can import project modules, then loads default configuration and paths using get_settings, and defines CLI arguments such as catalogue CSV location, output directory for artifacts, model name, batch size, and index version, with environment variables as overrides. After parsing the arguments, it resolves catalogue and output paths to absolute paths, creates a MatcherConfig object, optionally updates its model_name from the --model argument, and sets its index_version from --index-version. Finally, it calls build_index() with the parameters to actually read the catalogue, encode items, and write the index files to disk, and this whole process is triggered when the script is run as a main program.

evaluate.py

This script is a command-line tool that evaluates how well the semantic matcher retrieves the correct items for a set of labeled queries using Recall@k and Mean Reciprocal Rank (MRR). It parses arguments (or environment-based defaults) for the path to a CSV of queries, the artifacts directory containing the prebuilt index, the cut off top_k, an optional limit on how many queries to evaluate, and the index version to load. After resolving the queries and artifacts paths, it reads the queries CSV into a DataFrame, optionally truncates it to max_queries, and loads a SemanticMatcher instance from the artifacts directory for the given index version. For each query row, it calls matcher.match(query_text, top_k), stores the ranked list of returned item_ids, and records the true item_id, then computes Recall@1, Recall@5 (or up to top_k), and MRR from these ranked lists and ground truths, finally printing the three metric values to summarize model performance.

fit_calibration.py

This script is a command-line tool that learns calibration parameters for the semantic matcher so its similarity scores can be mapped to better-behaved confidence values. It loads configuration and paths from settings and CLI arguments (queries file, artifacts directory, negatives per query, max queries, random seed, and index version), then opens the corresponding versioned artifacts folder to read the matcher config, embedding matrix, and item ID list. Using the model specified in the config, it encodes each query (after text normalization), looks up the embedding of the correct catalog item to collect positive dot-product scores, and samples several random other items per query to collect negative scores. It concatenates these positive and negative scores, builds binary labels (1 for positives, 0 for negatives), fits a sigmoid with fit_sigmoid to obtain parameters a and b, writes these back into the matcher configuration as calibration_a and calibration_b, saves the updated config, and prints the new calibration parameters so the system can later convert raw similarity scores into calibrated confidence estimates.

calibration.py

This code implements a simple sigmoid-based calibration for model scores by learning two parameters, a and b, with gradient descent. The sigmoid function safely computes the logistic function in a numerically stable way for positive and negative inputs, while calibrate_score applies the learned linear transform a*score + b and then passes it through the sigmoid to turn a raw similarity score into a probability-like value. The fit_sigmoid function takes a list of scores and binary labels, initializes a and b, and repeatedly updates them using gradient descent on the logistic loss: for each score–label pair it computes a prediction with the current parameters, accumulates gradients for a and b, averages them over all samples, adds optional L2 regularization on a, and then steps the parameters in the opposite direction of the gradient for a fixed number of iterations, finally returning the fitted a and b.

config.py

This code defines a configuration dataclass for the semantic matcher, specifying all key parameters needed to run the model. The MatcherConfig class holds defaults such as the embedding model_name, how many semantic and lexical candidates to retrieve (semantic_top_n, lexical_top_n), the mixing weight between semantic and lexical scores (hybrid_alpha), score thresholds for triggering lexical fallback (semantic_threshold, lexical_threshold), calibration parameters calibration_a and calibration_b used to turn raw scores into calibrated confidences, and an index_version string to track which index artifacts are used. The from_json class method loads these settings from a JSON file into a MatcherConfig instance, to_json saves the current configuration back to disk as JSON (pretty-printed and key-sorted), and as_dict returns the configuration as a plain Python dictionary for easier inspection or logging.

index.py

This code builds and saves a complete hybrid search index (semantic + lexical) from the product catalogue CSV. It first defines build_document, which concatenates key text fields from each catalogue record (name, brand, model, category, description, etc.) into a single string, and _write_jsonl, which writes records line-by-line as JSON. The build_index function then loads the CSV, creates a list of normalized texts and corresponding item dictionaries (adding a normalized_text field), and initializes or updates a MatcherConfig with the chosen model name and index version. Using a SentenceTransformer model, it encodes all texts into embeddings, converts them to float32, and builds a FAISS inner-product index, writing both the FAISS index and the raw embeddings to disk in a versioned folder. In parallel, it tokenizes each text, fits a BM25Okapi lexical index and pickles it, saves the list of item IDs, the full items as a JSONL file, and the config and small metadata JSON (number of items and embedding dimension), returning the updated configuration so the retrieval system can later load these artifacts and serve hybrid semantic search.

metrics.py

This code defines two common ranking metrics that evaluate how well a system's ranked results match the true item. The recall_at_k function takes a list of ranked result lists and a list of ground-truth IDs, for each query it checks whether the true item appears within the top k positions of that query's ranked list, counts how many queries are hits, and returns the fraction of queries with the correct item in the top k. The mean_reciprocal_rank function also iterates over ranked lists and truths, finds the rank (position) of the true item in each list (if present), adds the reciprocal of that rank 1 / rank to a running total, and finally divides by the number of queries, producing the average reciprocal rank across all queries as a single performance score.

retrieval.py

This code implements the core semantic matcher that loads a hybrid index and answers search queries with scored, calibrated, and explained results. The MatchResult dataclass is a container for a single match (item ID, official name, final score, calibrated confidence, explanation string, and full record) with a helper as_dict method to convert it to a JSON-friendly dictionary. The SemanticMatcher class holds a FAISS semantic index, a list of item records, a BM25 lexical index, a SentenceTransformer model, and a MatcherConfig, and caches each item's normalized text for later token overlap explanations. The from_artifacts class method chooses the right versioned artifacts directory (either by explicit index version or by taking the latest), loads config, FAISS index, BM25 model, items JSONL, and the sentence-transformer model, then returns a ready-to-use matcher instance. The match method normalizes and embeds the query, runs FAISS to get semantic candidates and normalizes their scores to [0,1], runs BM25 on tokenized query to get lexical candidates and normalizes them by the maximum BM25 score, then decides whether to use hybrid scoring or a lexical fallback based on configured semantic and lexical thresholds. For the union of semantic and lexical candidate indices, it computes a final score (weighted mix or pure lexical), calibrates it into a confidence via calibrate_score, builds an explanation containing the mode and overlapping tokens between the query and item text, wraps everything into MatchResult objects, sorts them by score in descending order, and returns the top-k results as dictionaries.

settings.py

This code defines an application settings class that centralizes configuration for file paths and index version, with automatic loading from .env file. The AppSettings class inherits from BaseSettings, and its model_config tells Pydantic to read a .env file, then exposes fields for catalogue_path, queries_path, artifacts_dir, and index_version, each with sensible defaults. The versioned_artifacts_dir property constructs the full path to a specific version of the artifacts directory by combining artifacts_dir and index_version (For example artifacts/v1.0.0). The resolve_path method converts a relative path into an absolute one rooted at the project's top-level directory (three levels above this file), but returns the path unchanged if it

is already absolute. Finally, get_settings() is a small helper that instantiates and returns AppSettings, triggering Pydantic's environment-driven loading behaviour.

text_utils.py

This code implements basic text cleaning and token utilities for the semantic matcher. The noise_tokens set lists words (in english and french) related to price or buying that should be removed from text. The strip_accents function normalizes unicode text and drops combining accent characters so, for example, "café" becomes "cafe". The normalize_text function handles none as empty, lowercases text, removes accents, replaces any non-alphanumeric characters with spaces, splits into tokens, and filters out empty tokens and any in noise_tokens, finally joining the remaining tokens back into a normalized string. The tokenize function simply returns the list of tokens produced by normalize_text. The join_fields function concatenates multiple field values into a single string, skipping none, empty strings, and "nan", which is useful for building a clean document text from several metadata fields.

Tests et validation du système

test_calibration.py

- These tests are just checking that two functions behave correctly:
  - ✓ sigmoid (turns a number into value between 0 and 1).
  - ✓ fit_sigmoid (learns two numbers a and b so that sigmoid(a*score + b) fits given labels)
- test_sigmoid_zero
  - ✓ When you put 0 into sigmoid, the result should be 0.5
- test_sigmoid_positive
  - ✓ If you give a positive number (1 or 10), the result should be more than 0.5, and for 10 it should be very close to 1.
- test_sigmoid_negative
  - ✓ If you give a negative number (-1 or -10), the result should be less than 0.5, and for -10 it should be very close to 0.
- test_fit_sigmoid_empty
  - ✓ If there is no data (empty lists), fit_sigmoid should not crash and should return default values a = 1.0, b = 0.0.
- test_fit_sigmoid_perfect_separation
  - ✓ When scores perfectly separate 1s and 0s (big scores for label 1, small for label 0), fit_sigmoid should run and give some valid numbers for a and b (not None, not errors).

- test_fit_sigmoid_deterministic
  - ✓ If you run fit_sigmoid twice on the same data with the same settings, you should get (almost) the same a and b both times. This checks that the function is stable and not random.
- test_fit_sigmoid_all_ones
  - ✓ When all labels are 1, fit_sigmoid should still return valid numbers and not break.
- test_fit_sigmoid_all_zeros
  - ✓ When all labels are 0, fit_sigmoid should also return valid numbers and not break.
- test_fit_sigmoid_small_example
  - ✓ With two scores: 0.8 (label 1) and 0.2 (label 0), after fitting:
    - a should be positive.
    - The predicted value for 0.8 should be higher than the predicted value for 0.2.
  - ✓ This checks that fitting makes good scores get higher probabilities than bad ones.

test_integration.py

- These tests check that all the pieces (index building + semantic matcher) work together correctly on small, fake catalogues.
- Before the tests, two environment variables are set so that the transformers library uses only PyTorch and not TensorFlow, avoiding unwanted backends.
- test_semantic_matcher_integration
  - ✓ Creates a temporary folder and writes a small CSV catalogue with three smartphones (iPhone, Galaxy, Pixel) and their metadata.
  - ✓ Calls build_index() to build the semantic + lexical index artifacts from this CSV into the temp artifacts directory.
  - ✓ Loads a SemanticMatcher from those artifacts and runs a query "iPhone 13" with top_k=3.
  - ✓ Checks that:
    - The number of results is between 1 and 3.
    - The first result's item_id is "item1" (the iPhone row), meaning the matcher retrieves the correct top item.
    - The first result contains the keys "item_id", "score", and "confidence" so the output schema is correct.

- test_semantic_matcher_exact_match
  - ✓ Creates another temporary folder and a small CSV with two laptops: "MacBook Pro" and "Dell XPS".
  - ✓ Builds an index from this catalogue, then loads a SemanticMatcher.
  - ✓ Runs a query "MacBook Pro" with top_k=1.
  - ✓ Checks that there is at least one result and that the top result has item_id == "laptop1", confirming the matcher can return an exact match as the first result when the query matches a product name.

test_metrics.py

- These tests check that the two ranking metrics, recall_at_k and mean_reciprocal_rank, behave correctly in different simple scenarios.
- test_recall_at_k_perfect
  - ✓ Two queries, and in both cases the true item ("a" and "x") is at rank 1.
  - ✓ Checks that recall_at_k is 1.0 for both k=1 and k=3, meaning 100% of queries have the correct item in the top-k.
- test_recall_at_k_partial
  - ✓ Three queries, but the third truth "p" is never present in the results.
  - ✓ Only 2 out of 3 queries contain the correct item, so recall is 2/3 for both k=1 and k=3.
- test_recall_at_k_none
  - ✓ Two queries where none of the truths ("z", "w") appears in the result lists.
  - ✓ Checks that recall is 0.0 when there are no hits at all.
- test_recall_at_k_empty_results
  - ✓ No queries and no results.
  - ✓ recall_at_k should safely return 0.0 instead of crashing.
- test_recall_at_k_k_larger_than_results
  - ✓ k=5 but the result lists are shorter.
  - ✓ Both truths are present in the available results, so recall should still be 1.0, confirming that a large k does not break the function.
- test_mean_reciprocal_rank_perfect
  - ✓ In both queries, the true item is at rank 1.
  - ✓ MRR is 1.0 because the reciprocal rank is 1.0 for both queries.

- test_mean_reciprocal_rank_second_position
  - ✓ First truth "a" is at position 2 (reciprocal rank 1/2), second truth "x" is at position 1 (reciprocal rank 1).
  - ✓ MRR is the average (1/2+1)/2
- test_mean_reciprocal_rank_third_position
  - ✓ First truth "a" is at position 3 (reciprocal rank 1/3), second truth "x" is at position 1 (reciprocal rank 1).
  - ✓ MRR is (1/3+1)/2
- test_mean_reciprocal_rank_not_found
  - ✓ First truth "a" is not in the list (reciprocal rank 0), second truth "x" is at position 1 (reciprocal rank 1).
  - ✓ MRR is (0+1)/2
- test_mean_reciprocal_rank_empty_results
  - ✓ No queries and no results.
  - ✓ MRR should return 0.0 safely.
- test_mean_reciprocal_rank_mixed
  - ✓ Three queries:
    - "a" at position 1 → reciprocal rank 1.0
    - "d" at position 2 → reciprocal rank 0.5
    - "g" not found → reciprocal rank 0.0
  - ✓ MRR is the average (1+0.5+0)/3

test_text_utils.py

- These tests check that the text cleaning functions normalize_text and tokenize work as expected on many small examples.
- test_normalize_text_basic
  - ✓ Confirms that normalizing "Hello World" makes it lowercase and keeps spacing: "hello world".
- test_normalize_text_with_accents
  - ✓ Checks that accents are removed properly:
    - "Café" → "cafe"
    - "naïve" → "naive"
    - "résumé" → "resume"

- test_normalize_text_removes_special_chars
  - ✓ Ensures punctuation and symbols are removed or turned into spaces:
    - "Hello, World!" → "hello world"
    - "test@example.com" → "test example com"
    - "item-123" → "item 123"
- test_normalize_text_lowercase
  - ✓ Verifies everything is lowercased, even if the original has mixed or upper case.
- test_normalize_text_removes_noise_tokens
  - ✓ Checks that “noise” words like buy, price, acheter, prix are removed:
    - "buy price" becomes an empty string.
    - "hello price world" keeps only meaningful words: "hello world".
- test_normalize_text_none
  - ✓ If the input is None, the function should safely return an empty string instead of failing.
- test_normalize_text_empty
  - ✓ Empty string or just spaces should normalize to an empty string.
- test_tokenize_basic
  - ✓ Tokenizing "Hello World" should give the list ["hello", "world"] (lowercased, split into words).
- test_tokenize_with_special_chars
  - ✓ Tokenizing "Hello, World!" should ignore punctuation and still return ["hello", "world"].
- test_tokenize_removes_noise
  - ✓ Noise words are removed at token level too:
    - "buy price" → [] (no tokens).
    - "hello price world" → ["hello", "world"].
- test_tokenize_empty
  - ✓ Empty string or spaces should produce an empty list of tokens.

```
PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search> python -m pytest
============================= test session starts ==============================
platform win32 -- Python 3.12.7, pytest-8.3.5, pluggy-1.5.0
rootdir: C:\Users\safwe\Desktop\Project 2 NLP Sementic Search
configfile: pytest.ini
plugins: anyio-4.6.2.post1, Faker-33.0.0, langsmith-0.3.18, docker-3.1.2, typeguard-4.4.4
collected 33 items

tests\test_calibration.py .........                                      [ 27%]
tests\test_integration.py ..                                             [ 33%]
tests\test_metrics.py ...........                                        [ 66%]
tests\test_text_utils.py ...........                                     [100%]

============================== 33 passed in 22.27s ==============================
PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search>
```

I ran pytest, and all 33 tests passed successfully !

Linting & formatting

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(.venv) PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search> ruff check src/ tests/
F401 [*] `typing.Sequence` imported but unused
 --> src\semantic_matcher\index.py:6:52
  |
4 | import os
5 | import pickle
6 | from typing import Dict, Iterable, List, Optional, Sequence
  |                                                    ^^^^^^^^
7 |
8 | import faiss
  |
help: Remove unused import: `typing.Sequence`

F401 [*] `typing.Iterable` imported but unused
 --> src\semantic_matcher\metrics.py:3:20
  |
1 | from __future__ import annotations
2 |
3 | from typing import Iterable, List, Sequence
  |                    ^^^^^^^^
  |
help: Remove unused import

F401 [*] `typing.List` imported but unused
 --> src\semantic_matcher\metrics.py:3:30
```

```
(.venv) PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search> ruff check src/ tests/
All checks passed!
(.venv) PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search>
```

In this section, I started by using Ruff to clean the code. Ruff is a Python linter that scans source code for issues like unused imports, styling problems, and bugs to keep the code clean and consistent

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(.venv) PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search> black src/ tests/
reformatted C:\Users\safwe\Desktop\Project 2 NLP Sementic Search\src\semantic_matcher\__init__.py
reformatted C:\Users\safwe\Desktop\Project 2 NLP Sementic Search\src\semantic_matcher\metrics.py
reformatted C:\Users\safwe\Desktop\Project 2 NLP Sementic Search\src\semantic_matcher\settings.py
reformatted C:\Users\safwe\Desktop\Project 2 NLP Sementic Search\tests\test_text_utils.py
reformatted C:\Users\safwe\Desktop\Project 2 NLP Sementic Search\tests\test_calibration.py
reformatted C:\Users\safwe\Desktop\Project 2 NLP Sementic Search\src\semantic_matcher\index.py
reformatted C:\Users\safwe\Desktop\Project 2 NLP Sementic Search\tests\test_metrics.py
reformatted C:\Users\safwe\Desktop\Project 2 NLP Sementic Search\tests\test_integration.py
reformatted C:\Users\safwe\Desktop\Project 2 NLP Sementic Search\src\semantic_matcher\retrieval.py

All done! ✨ 🍰 ✨
9 files reformatted, 4 files left unchanged.
(.venv) PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search>
```

Then, I used Black, which is a Python code formatter, to automatically rewrite the files. This ensures a consistent style (spacing, etc…) and removes the need to format code by hand

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(.venv) PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search> isort src/ tests/
Fixing C:\Users\safwe\Desktop\Project 2 NLP Sementic Search\src\semantic_matcher\text_utils.py
Fixing C:\Users\safwe\Desktop\Project 2 NLP Sementic Search\tests\test_calibration.py
Fixing C:\Users\safwe\Desktop\Project 2 NLP Sementic Search\tests\test_metrics.py
(.venv) PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search>
```

Here, I used the isort tool, which automatically sorts and groups the import statements

```
(.venv) PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search> mypy src/ --ignore-missing-imports
src\semantic_matcher\retrieval.py:109: error: "object" has no attribute "get_scores"  [attr-defined]
src\semantic_matcher\retrieval.py:143: error: Argument 2 to "_token_overlap" has incompatible type "object"; expected "str"  [arg-type]
Found 2 errors in 1 file (checked 8 source files)
```

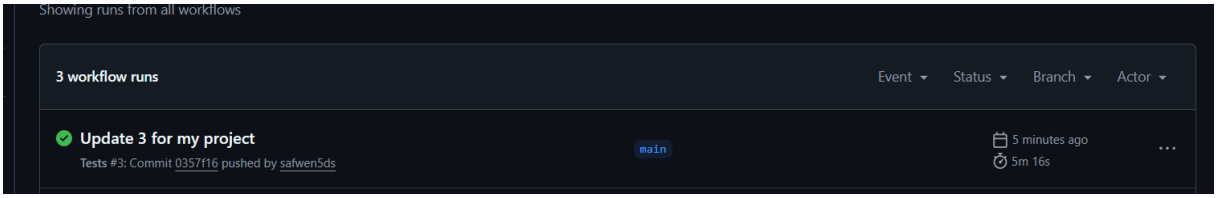
```
(.venv) PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search> mypy src/ --ignore-missing-imports
Success: no issues found in 8 source files
(.venv) PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search>
```

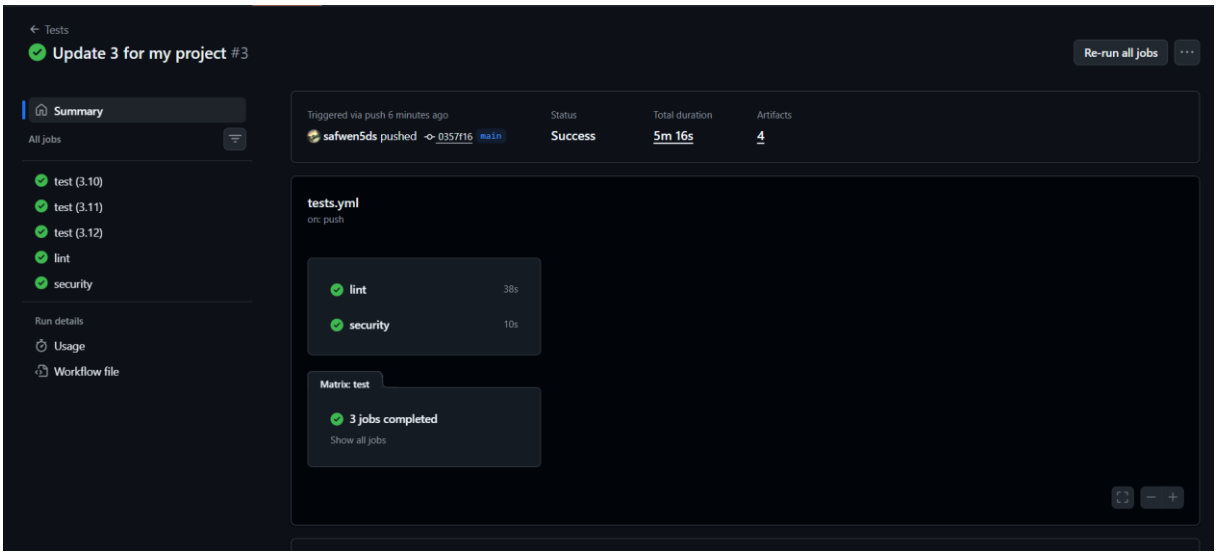
After using isort, I used mypy, which reads type hints and analyzes the code without running it to catch type errors early. The command I ran checks all typed code under src/ and reports where the actual types do not match the type hints, which is why we see the red errors

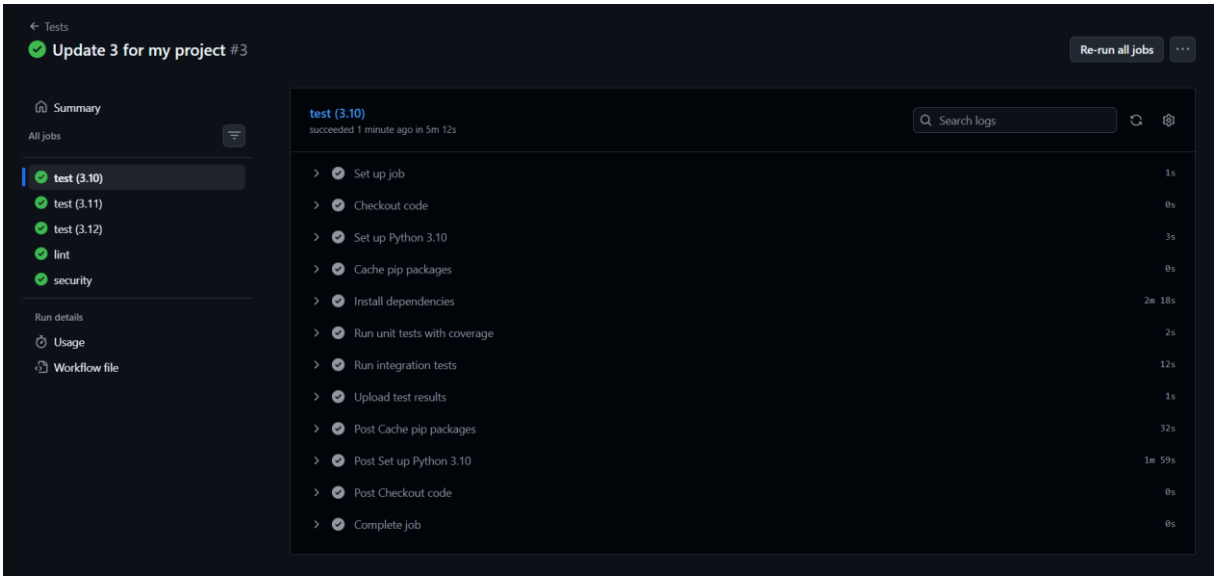
```
PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search> .\.venv\Scripts\activate
(.venv) PS C:\Users\safwe\Desktop\Project 2 NLP Sementic Search> bandit -r src/
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.10.11
Run started:2026-01-09 23:12:48.708178+00:00

Test results:
>> Issue: [B403:blacklist] Consider possible security implications associated with pickle module.
   Severity: Low   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.2/blacklists/blacklist_imports.html#b403-import-pickle
   Location: src/semantic_matcher\index.py:5:0
4       import os
5       import pickle
6       from typing import Dict, Iterable, List, Optional

--------------------------------------------------
>> Issue: [B403:blacklist] Consider possible security implications associated with pickle module.
   Severity: Low   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.2/blacklists/blacklist_imports.html#b403-import-pickle
   Location: src/semantic_matcher\retrieval.py:5:0
4       import os
5       import pickle
6       from dataclasses import dataclass

--------------------------------------------------
```

I used Bandit, a security linter, to perform a security check on the codebase. Bandit flagged issues B301/B403 regarding the use of the Python pickle module, noting that unpickling data from untrusted sources can lead to CWE-502 (“Deserialization of Untrusted Data”) and potential remote code execution (RCE). In this project, however, pickle is used solely to persist the BM25Okapi index (bm25.pkl), which is generated locally by the build_index pipeline and stored within a controlled output_dir. No user input, external uploads, or network-sourced data are ever deserialized. Because the .pkl files are created and consumed exclusively by this application within a trusted environment, the typical deserialization attack vector does not apply. Consequently, the security risk is considered low.

GitHub Actions CI workflow







- So that I can clearly explain what we did inside GitHub, let's first define what a GitHub Actions workflow is :

> A workflow is an automated process that runs on GitHub servers.

- I created a YAML file to test the code, check code quality, and check basic security issues. This workflow runs automatically when the code changes.

```yaml
on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]
```

- The workflow runs when a user pushes code to the main branch and when they open or update a pull request to the branch, so every change is checked automatically.

- The workflow is divided into 3 jobs.

- The first job is “test”, and it is responsible for running tests.

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.10", "3.11", "3.12"]
```

- All tests run on an Ubuntu machine.

- The same tests are executed with three Python versions : This checks that the project works correctly on different environments.

```yaml
    steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Python ${{ matrix.python-version }}
      uses: actions/setup-python@v5
      with:
        python-version: ${{ matrix.python-version }}
        cache: 'pip'
```

- The first step in the test job is checkout code, where the repository code is downloaded to the runner.
- The second step is setup Python, so the required Python version from the matrix is installed.

```yaml
    - name: Cache pip packages
      uses: actions/cache@v4
      with:
        path: ~/.cache/pip
        key: ${{ runner.os }}-pip-${{ matrix.python-version }}-${{ hashFiles('**/requirements.txt') }}
        restore-keys: |
          ${{ runner.os }}-pip-${{ matrix.python-version }}-
          ${{ runner.os }}-pip-
```

- The third step is cache pip packages, where downloaded Python packages are stored in a cache. This avoids reinstalling them every time and makes the workflow faster.

```
- name: Install dependencies
  run: |
    python -m pip install --upgrade pip setuptools wheel
    pip install -r requirements.txt
    pip install pytest pytest-cov pytest-xdist pytest-timeout
```

- Step four is install dependencies.
  All project dependencies are installed, plus testing tools like: pytest for running tests, pytest-cov for coverage, and pytest-timeout to prevent infinite tests.

```
- name: Run unit tests with coverage
  env:
    TRANSFORMERS_NO_TF: "1"
    USE_TF: "0"
    PYTHONPATH: ${{ github.workspace }}/src
  run: |
    pytest tests/test_text_utils.py tests/test_metrics.py tests/test_calibration.py \
      -v \
      --cov=src/semantic_matcher \
      --cov-report=html \
      --cov-report=term-missing \
      --tb=short \
      --strict-markers \
      --timeout=300
```

- Step five is run unit tests with coverage, so unit tests check small parts of the code separately, and code coverage shows which parts of the code are tested and which are not.

```
- name: Run integration tests
  env:
    TRANSFORMERS_NO_TF: "1"
    USE_TF: "0"
    PYTHONPATH: ${{ github.workspace }}/src
  run: |
    pytest tests/test_integration.py \
      -v \
      --tb=short \
      --strict-markers \
      --timeout=600
```

- Step six is run integration tests, so integration tests check how different parts of the project work together, not just individual functions.

```yaml
- name: Upload test results
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: test-results-${{ matrix.python-version }}
    path: |
      htmlcov/
      .coverage
    retention-days: 30
```

- Step seven is upload test results, so test reports and coverage files are uploaded as artifacts. They can be downloaded later from GitHub.

```yaml
lint:
  runs-on: ubuntu-latest
  steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Set up Python
    uses: actions/setup-python@v5
    with:
      python-version: "3.11"
      cache: 'pip'

  - name: Install linting tools
    run: |
      python -m pip install --upgrade pip
      pip install ruff black isort mypy
```

Now the second job “linter” checks code quality and style, not functionality.

It uses these tools that we talked about earlier :

- ruff: finds code errors and bad practices
- black: checks code formatting
- isort: checks import order
- mypy: checks type hints

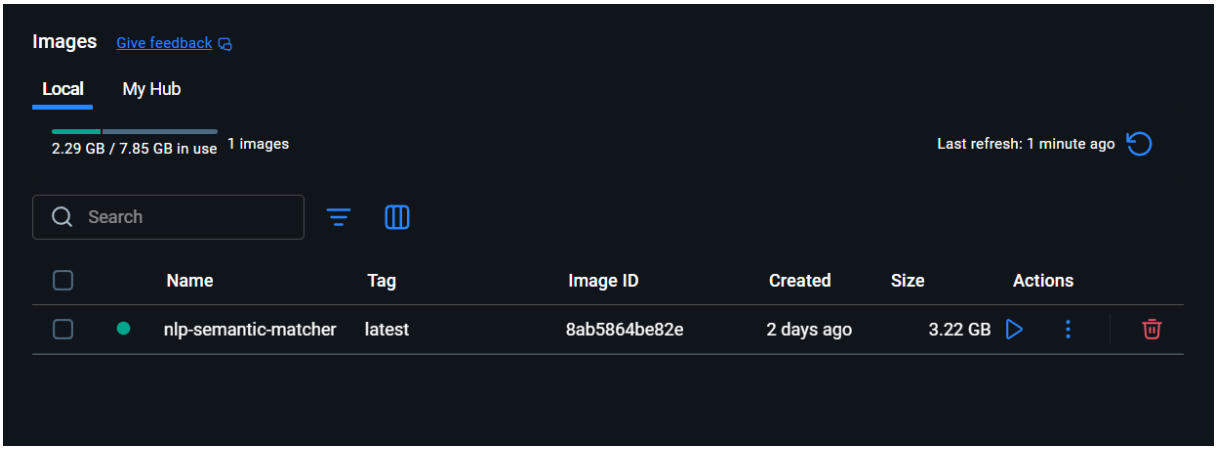
- Errors do not stop the workflow : They are reported as warnings instead of failures.

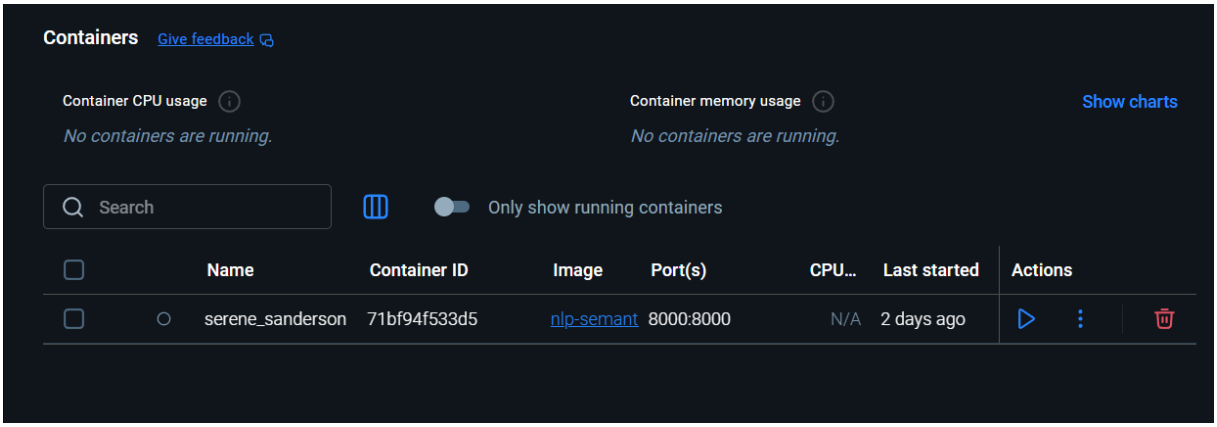
```yaml
security:
  runs-on: ubuntu-latest
  steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Set up Python
    uses: actions/setup-python@v5
    with:
      python-version: "3.11"

  - name: Install dependencies
    run: |
      python -m pip install --upgrade pip
      pip install bandit[toml]
```

- Job 3 is security

- This job performs a basic security analysis
- Tool used: Bandit

Docker File





Let's move to the most important part, which is setting up the Docker container and to do that, we created a Dockerfile

```
FROM python:3.11-slim AS builder
```

We started off by choosing our base image

```
WORKDIR /build
```

Next, we defined the working directory

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    && rm -rf /var/lib/apt/lists/*
```

Then we handled the system dependencies

```
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
```

To keep the environment isolated, I created a Python virtual environment

```
COPY requirements.txt .
```

Next, I copied the dependency list

```
RUN pip install --no-cache-dir torch --index-url https://download.pytorch.org/whl/cpu
RUN pip install --no-cache-dir -r requirements.txt
```

We installed PyTorch first because it's large and specific and after that we installed the rest of the project's dependencies

```
COPY src/ ./src/
COPY scripts/ ./scripts/
```

We then copied in the project's main code and scripts

```
ARG CATALOGUE_PATH=catalogue.csv
ARG QUERIES_PATH=queries.csv
ARG ARTIFACTS_DIR=artifacts
ARG MODEL_NAME=sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
ARG INDEX_VERSION=1.0.0
```

I defined build-time arguments to make the setup flexible

```
ENV HF_HOME=/build/cache/huggingface
ENV ARTIFACTS_DIR=/build/${ARTIFACTS_DIR}
ENV INDEX_VERSION=${INDEX_VERSION}
ENV MODEL_NAME=${MODEL_NAME}
ENV PYTHONPATH=/build/src
```

We then moved on to setting important environment variables

```
COPY ${CATALOGUE_PATH} ./catalogue.csv
COPY ${QUERIES_PATH} ./queries.csv
```

Next, I copied the datasets

```
RUN mkdir -p ${ARTIFACTS_DIR} ${HF_HOME}
```

Here , we prepared the necessary directories for the artifacts and the hugging face cache

```
RUN python scripts/build_index.py \
    --catalogue ./catalogue.csv \
    --output ${ARTIFACTS_DIR} \
    --index-version ${INDEX_VERSION} \
    --model ${MODEL_NAME}
```

We run this script to build the index from the catalogue data using the specified model

```
RUN python scripts/fit_calibration.py \
    --queries ./queries.csv \
    --artifacts ${ARTIFACTS_DIR} \
    --index-version ${INDEX_VERSION}
```

Then we fine-tuned and calibrated the system using the queries data

```
FROM python:3.11-slim AS runtime
```

After finishing the build, we switched to a cleaner and lighter runtime image

```
WORKDIR /app
```

We set up the working directory again. *This is where the final application lives and runs*

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/*
```

I installed minimal runtime utilities

```
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
```

I brought over the virtual environment

```
COPY --from=builder /build/artifacts ./artifacts
```

I also transferred the previously built artifacts

```
ENV HF_HOME=/root/.cache/huggingface
COPY --from=builder /build/cache/huggingface /root/.cache/huggingface
```

We copied over the cached model files to save time

```
COPY src/ ./src/
COPY api/ ./api/
```

Next, I brought in the app's source and API code

```
ENV ARTIFACTS_DIR=/app/artifacts
ENV INDEX_VERSION=1.0.0
ENV MODEL_NAME=sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
ENV PYTHONPATH=/app/src
```

Here , we set up the runtime configuration through environment variables

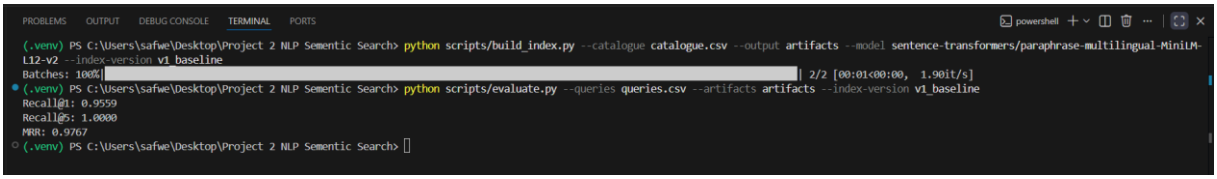
```
EXPOSE 8000
```

After that we exposed port 8000 for the API

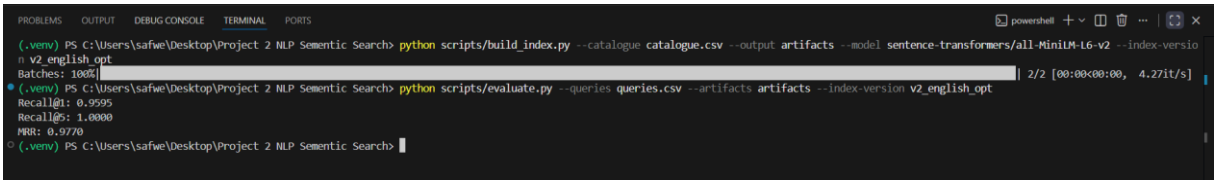
```
CMD ["uvicorn", "api.app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Finally, we defined the container's startup command

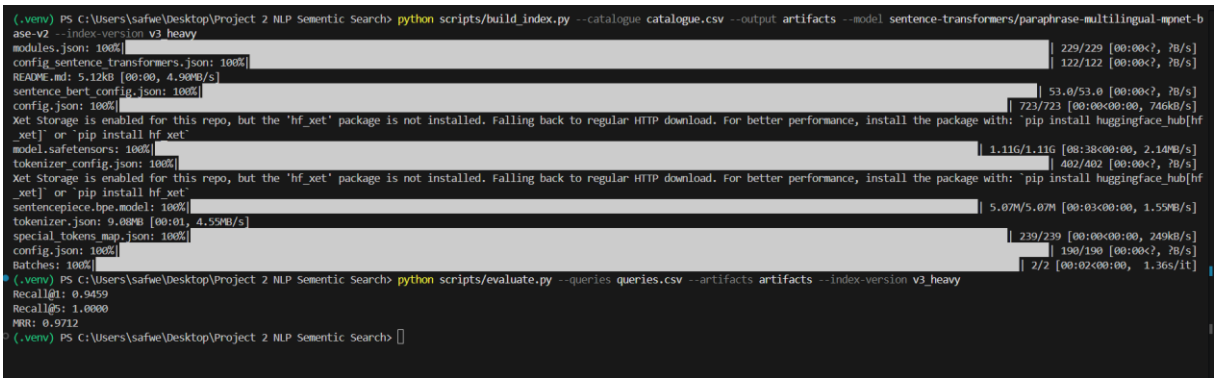
Experiments and Analysis



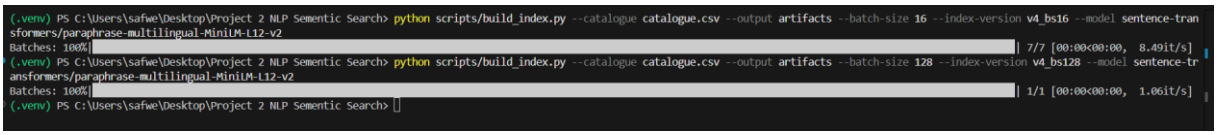
Exp #1 : In this part, let's start with testing our default model, our first baseline, which is the multilingual model (paraphrase-multilingual-MiniLM-L12-v2). The Recall@1 value means that for 95.59% of queries, the correct item appeared as the first result. The Recall@5 value means for all queries, the correct item appeared within the top 5 results. This is excellent for a search system. MRR (The Mean Reciprocal Rank) is very high (0.97), indicating that relevant results generally appear at high ranks



Exp #2 : Here we changed the model to an English-only model (all-MiniLM-L6-v2), and it outperformed the multilingual model slightly. Recall@5 = 99.95%, so almost perfect recall at the 5-result level. MRR = 0.9767, which is higher than the first baseline. This suggests that technical terms like MacBook Pro or Bosch are indeed universal enough that an English-only model can handle French descriptions effectively. So here, instead of using a multilingual model that consumes a lot of resources, we can save compute resources by using an English model



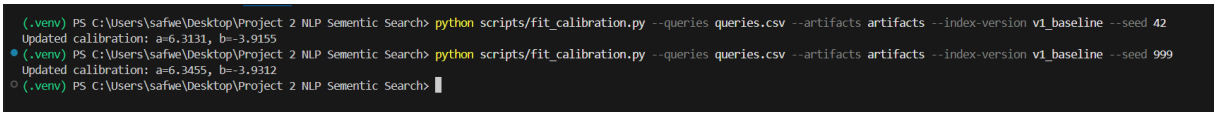
Exp #3 : Here we made an upgrade and we used a larger multilingual model (mpnet), and surprisingly it shows a slightly worse performance (less recall and less MRR than the baseline). So despite being more computationally expensive, the larger model did not deliver better results for our datasets. This suggests the smaller multilingual model is more efficient for our use case



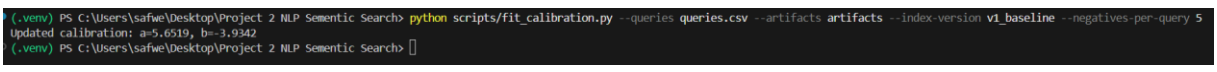
Exp #4 : In this section, we are comparing batch sizes. Contrary to expectations, the smaller batch size was significantly faster: we processed at 8.80 items per second with batch size = 16 and we processed at only 1.06 items per second with batch size = 128. This indicates that for my system, smaller batch sizes (16) provide better throughput, so the best batch depends on the hardware and the used dataset

Exp #5 : This experiment shows how result count affects user experience: top-1 results give 95.59% recall : it is good, but not perfect because users would need to look beyond the first result about 4.4% of the time. Top-5 results give 100% recall, so it is perfect recall and users will always find the right item within the first 5 results. The MRR increases from 0.9559 to 0.9767 when looking at more results

Exp #6 : Now let's start the experiments with the calibration. This experiment tests how the calibration parameters change when using different numbers of negative examples (incorrect matches) during training. With 1 negative per query: a=7.11, b=-3.53 and with 10 negatives per query: a = 7.29, b = -3.85. The calibration parameters changed only slightly between the two configurations. This indicates that the confidence scoring system is stable and not highly sensitive to the number of negative examples

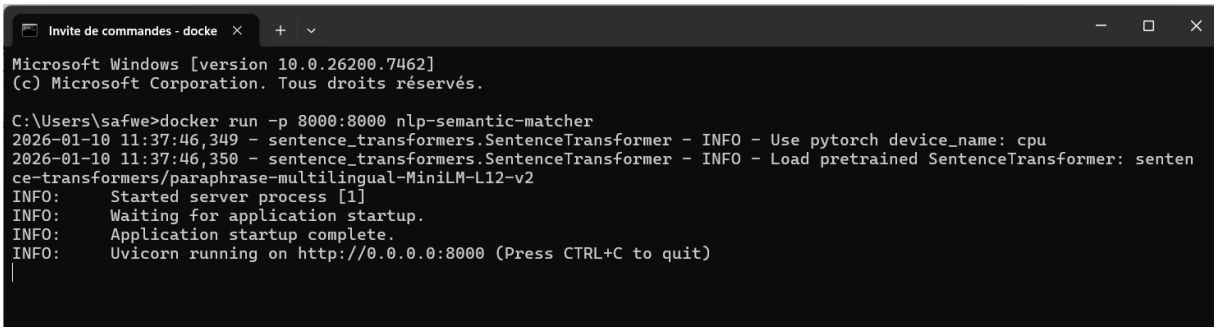


Exp #7 : This test checks if the results are consistent across different random seeds (Reproducibility Check), which is crucial for scientific validity. This demonstrates excellent reproducibility in the model's confidence scoring system. The minimal variation confirms that the results aren't due to random chance but reflect the true behavior of my semantic search system.

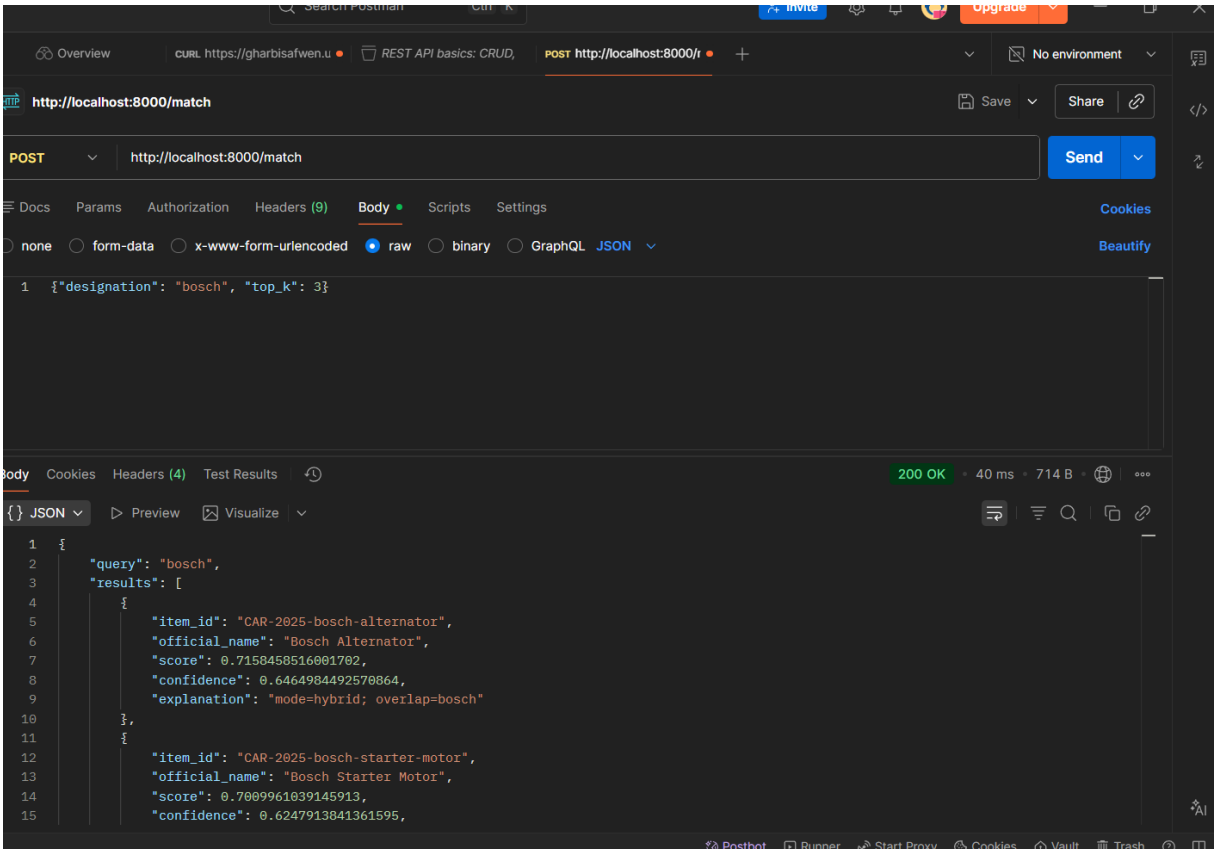


Exp #8 : This experiment provides the parameters needed to calculate confidence thresholds. Calibration parameters: a=5.65, b=-3.93 and Sigmoid formula: S(x) = 1/(1 + e^-(5.65x - 3.93)) To achieve 80% confidence (0.8), we need to solve for x in: 0.8 = 1/(1 + e^-(5.65x - 3.93)). This yields x = 0.83 (raw similarity score). This means any search result with a similarity score above 0.83 should be considered a confident match with approximately 80% probability of being correct.

Demo

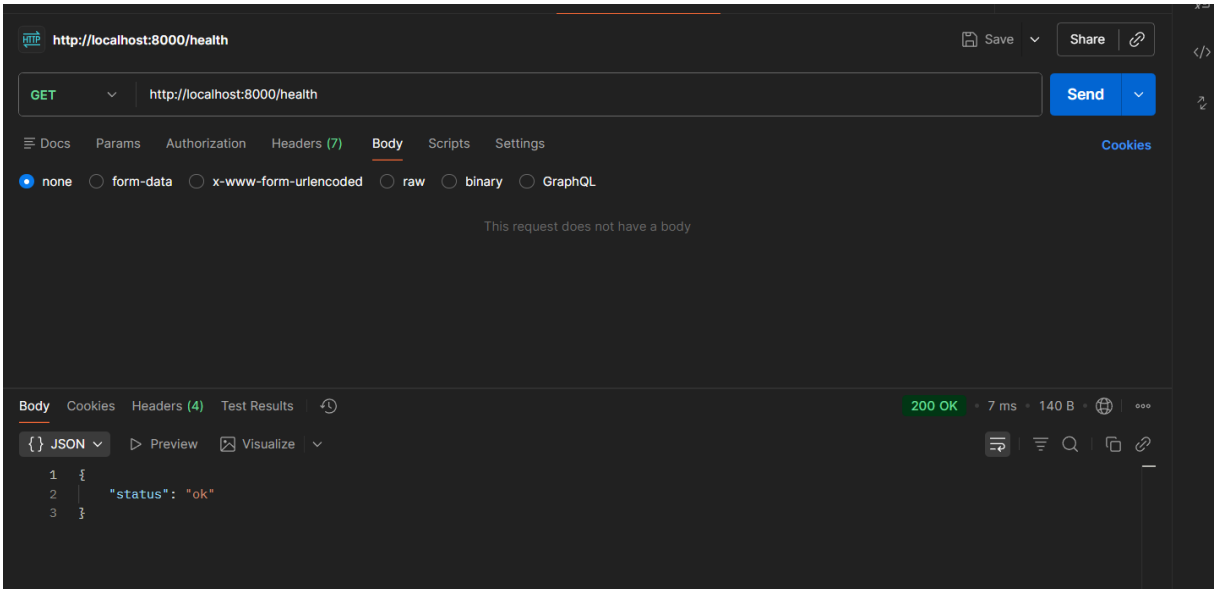


I have already built the app image using a Dockerfile (it took some time to download all the dependencies and build the image), and here I'm creating and running a new container using that image

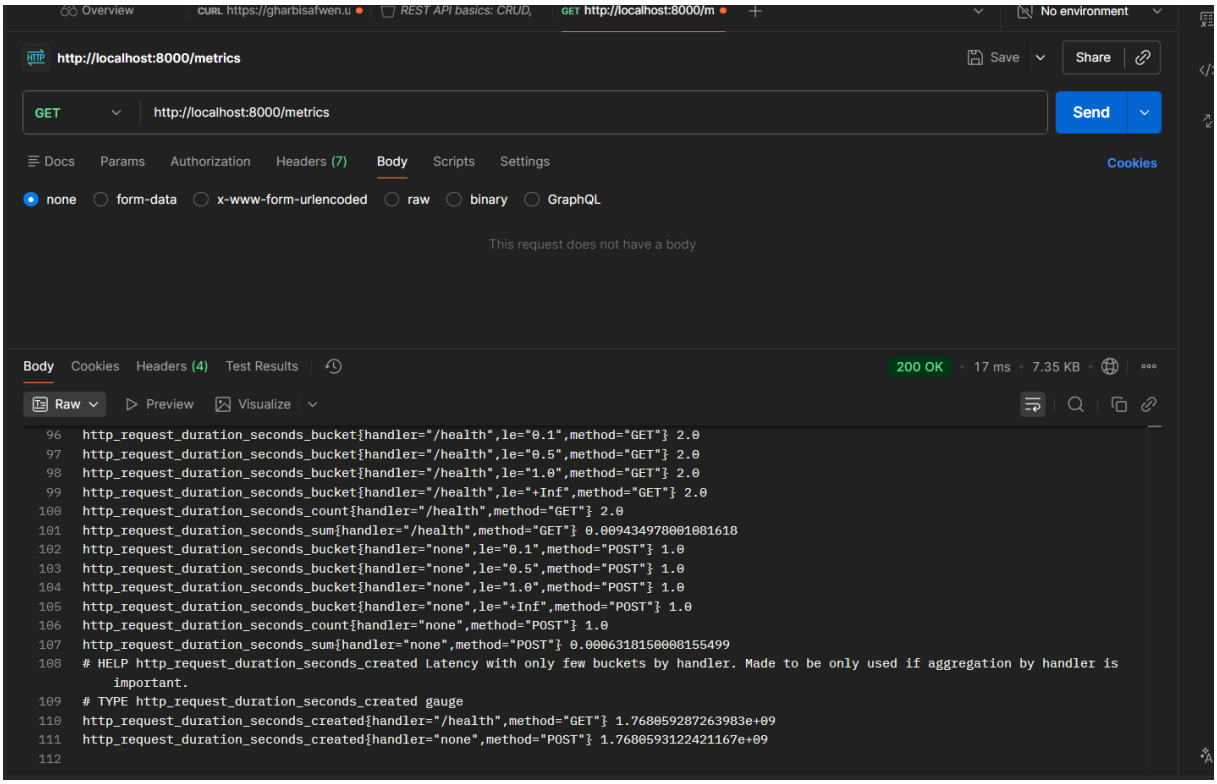


Here, I tested the API using Postman, and it is returning the top three results that match my query. We can see that each item contains:

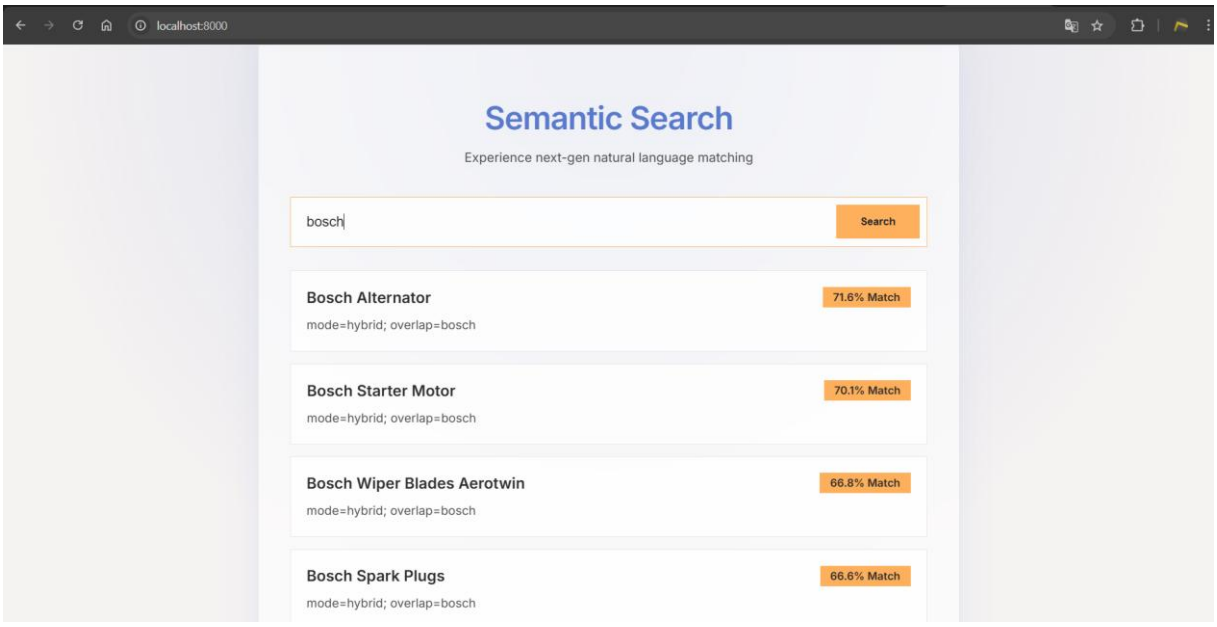
- ID: The unique identifier.
- Official Name: The name of the item as it appears in the database.
- Score: The similarity score (usually between 0 and 1) calculated using a hybrid of semantic (vector) and lexical (BM25) search.
- Confidence: A calibrated confidence score representing how reliable the match is.
- Explanation: A string explaining the matching logic specifically whether it used hybrid mode (semantic + lexical) or lexical_fallback (used if the semantic search fails) and which tokens overlapped

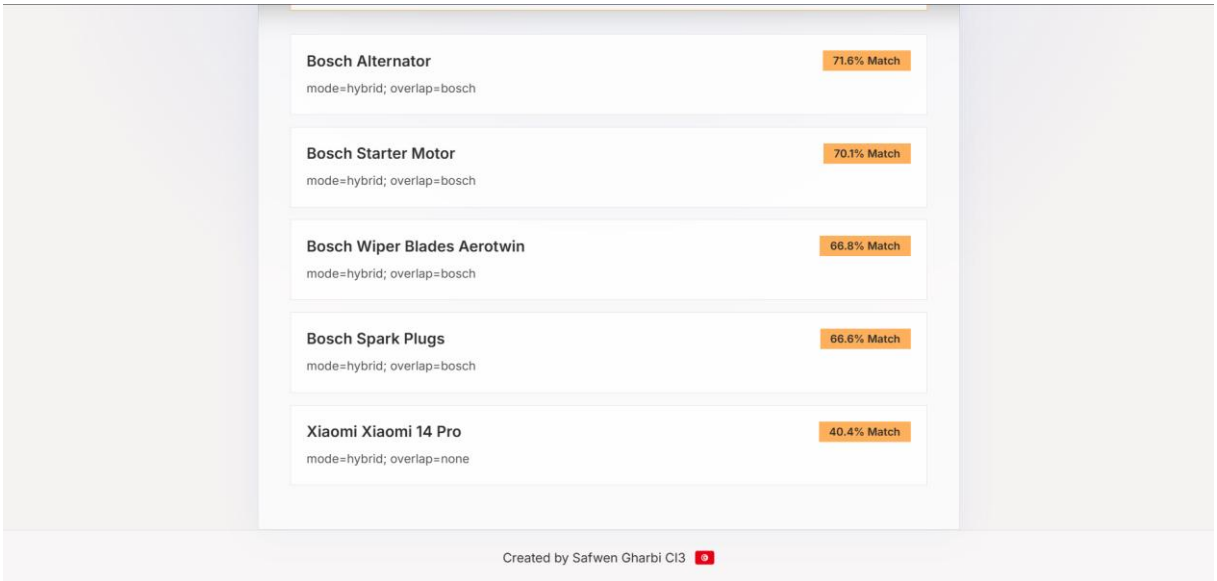


This is a health endpoint and its primary job is to confirm that the application is alive and the web server (Uvicorn) is responsive.



I made this endpoint to provides real-time performance data and operational statistics about the API. It allows me to monitor how many requests are coming in, which endpoints are most used, how long requests take to process (latency), and if there are any errors. It uses the prometheus library and this library automatically hooks into FastAPI to measure every request and its duration.





One last thing I added to the project, which wasn't mentioned in the code explanation, is an interface for the API. If you run the container and access localhost:8000, the UI for semantic search will open. You can search the database, and it will return the top 5 results that match your query

```
C:\Users\safwe>docker run -p 8000:8000 nlp-semantic-matcher
2026-01-10 15:45:14,933 - sentence_transformers.SentenceTransformer - INFO - Use pytorch device_name: cpu
2026-01-10 15:45:14,933 - sentence_transformers.SentenceTransformer - INFO - Load pretrained SentenceTransformer: senten
ce-transformers/paraphrase-multilingual-MiniLM-L12-v2
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     172.17.0.1:48748 - "GET / HTTP/1.1" 200 OK
Batches: 100%|██████████| 1/1 [00:00<00:00,  7.01it/s]
2026-01-10 15:45:58,094 - api.app - INFO - match_request: query_length=5, top_k=5, mode=hybrid, results_count=5, latency
_ms=148.85
INFO:     172.17.0.1:48762 - "POST /match HTTP/1.1" 200 OK
```

Every request is printed to the console, along with all its information and its latency

Conclusion

This project set out with the objective of overcoming the limitations of traditional keyword-based search by developing a robust Hybrid Semantic Search Engine. By combining the contextual understanding of Sentence-Transformers with the exact-matching precision of BM25, and optimizing the retrieval speed with FAISS, we successfully created a system capable of interpreting user intent rather than just matching text strings.